

Eye Gaze Tracking for Virtual Mouse Control Using Computer Vision

Aishwariya SK

Department of Artificial Intelligence
Kongu Engineering College
Perundurai, India
aishwariyask.22aid@kongu.edu

Elakkiya Dharshini A

Department of Artificial Intelligence
Kongu Engineering College
Perundurai, India
elakkiyasharshinia.22aid@kongu.edu

Harshini B

Department of Artificial Intelligence
Kongu Engineering College
Perundurai, India
Harshinib.22aid@kongu.edu

Harini M

Department of Artificial Intelligence
Kongu Engineering College
Perundurai, India
harini.ai@kongu.ac.in

Abstract— Traditional human–computer interaction (HCI) relies heavily on physical devices like the mouse and keyboard, which may not be accessible for individuals with motor impairments. Recent advances in computer vision and deep learning provide opportunities to create alternative, low-cost input methods using only a webcam. In this work, we present a real-time eye-gaze tracking system for virtual mouse control based on MediaPipe Face Mesh and blink detection. The system captures iris landmarks through a standard webcam, maps gaze movements to cursor motion, and uses different blink durations to trigger various mouse actions including left click, right click, double click, scrolling, and drag-and-drop using PyAutoGUI. A lightweight graphical interface using Tkinter is integrated to toggle movement, scrolling, and drag modes dynamically. The proposed solution achieves robust cursor control without additional hardware and demonstrates high responsiveness under normal lighting conditions. This approach has potential applications in assistive technologies, hands-free interaction, and accessibility-focused computing environments.

Keywords—Eye-gaze tracking, Human–computer interaction, Assistive technology, Blink detection, Virtual mouse, Computer vision, Accessibility, MediaPipe, PyAutoGUI, Tkinter

I. INTRODUCTION

Traditionally, physical input devices like keyboards, mouse, and touchscreens have been the mainstay of human–computer interaction (HCI). Although these devices work quite well for the majority of users, they have serious drawbacks for those with motor impairments or in circumstances when hands-free contact is necessary. The increasing demand for accessibility and inclusive computing environments has motivated the exploration of alternative input modalities that do not rely on conventional hardware.

Eye-gaze tracking has emerged as one of the most promising approaches. The human eye is an efficient and reliable indicator of attention, capable of producing rapid and precise movements that can be leveraged for input control. Unlike hand gestures or speech, gaze interaction is silent, requires minimal effort, and directly correlates with user intent in pointing or selection tasks. This makes it especially valuable in assistive technologies, where users may have limited control over other parts of the body.

However, the deployment of gaze-based systems faces persistent challenges. Conventional eye-tracking solutions often rely on specialized infrared sensors or proprietary headsets, which are costly and unsuitable for large-scale or home-based use. These systems also tend to suffer from calibration drift, lighting sensitivity, and reduced accuracy in unconstrained environments. As a result, while they are effective in research laboratories and controlled applications, they remain impractical for everyday computing scenarios.

Recent development in computer vision and machine learning, especially in facial landmark detection, have enabled accurate and real-time gaze estimation using standard webcams. Gaze tracking and blink detection can be combined to create inexpensive systems that can mimic key mouse functions like dragging, scrolling, clicking, and cursor movement. These devices could facilitate hands-free computer scenarios, increase accessibility for persons with physical limitations, and advance the field of assistive technologies.

The system described in this work leverages MediaPipe's Face Mesh model to capture eye and iris landmarks, combined with blink duration analysis for command execution. A virtual mouse control framework is implemented, offering real-time cursor control, mode switching, and gesture-based actions through blink signals. The approach prioritizes simplicity, low computational cost, and hardware independence, relying only on a standard webcam and general-purpose computing resources.

II. RELATED WORKS

Waideman, Bruno, and Plínio Thomaz Aquino [1] proposed an Augmentative and Alternative Communication (AAC) framework that integrates eye-tracking with word recommendation using language models. Methodologically, the system uses eye-tracking hardware to capture gaze input and a language model to suggest probable words. Performance evaluation shows that the integration of gaze with word prediction significantly speeds up communication. However, challenges such as system calibration, maintaining high accuracy under varying lighting conditions, and ensuring real-time responsiveness were noted as limitations. This study demonstrates the growing potential of combining gaze

tracking with AI-driven text recommendation for accessible human-computer interaction.

Lin, Yi, Xingwei Chen, Chuyang Zhang, and Jingjing Chen [2] introduced a novel method for manipulating virtual objects that combines head-eye coordination with eye-gaze selection. The system detects gaze points to select targets, while coordinated head movements enable manipulation, offering a hands-free interaction paradigm. The system achieved high responsiveness, making it suitable for VR and AR applications. However, challenges such as user fatigue due to prolonged head movements and calibration drift in gaze detection were noted. The study emphasizes the benefits of multi-modal integration, where combining gaze and head motion yields more robust performance than single-modal approaches.

Beevi, L. Sherin, PM Joe Prathap, and W. Vinil Dani [3] presented an innovative approach to mouse cursor control aimed at enhancing user interaction by replacing traditional hardware with computer vision methods. The methodology involves real-time face and eye tracking, with mapped gaze points directly linked to screen coordinates for pointer control. The authors reported satisfactory performance with relatively Challenges include environmental dependencies such as lighting conditions and background noise, which affect accuracy, as well as the system's need for individual calibration. However, the study adds to the expanding corpus of research.

Sai, K. Chenchu Naga, S. Shargunam, K. Surendra Reddy, K. Bheemendra, and K. Santhosh Reddy [4] developed a virtual mouse system that leverages multimodal inputs, specifically hand gestures, eye gaze, and speech recognition. The system integrates computer vision for gesture detection, eye-tracking techniques for pointer movement, and speech recognition for executing commands such as opening applications or confirming selections. Evaluation results showed that combining multiple modalities enhanced accuracy and reduced input ambiguity compared to single-modal systems. However, challenges were identified in synchronization between different input streams and in maintaining robust performance under noisy speech environments or poor lighting.

Dharavath, Krishna, Ganguru Manoj Kumar, Kallem Rishitha Reddy, and Mandla Hemanth Reddy [5] presented a virtual mouse that is controlled by gesture and voice designed for elderly users who may face difficulties operating conventional input devices. Their method combines speech recognition to carry out commands with computer vision to identify hand motions. The methodology relies on webcam-based gesture recognition to move the cursor and voice commands to perform clicks or open applications. Challenges included limited gesture recognition accuracy under poor lighting and reduced speech recognition reliability in noisy environments. The work highlights how multimodal interfaces can improve accessibility while tailoring design considerations to the specific needs of older populations.

Morajkar, Aarti, Atheena Mariyam James, Minoli Bagwe, Aleena Sara James, and Aruna Pavate [6] developed a vision- and speech-based virtual mouse system aimed at providing a low-cost and assistive solution for physically challenged

individuals. The methodology employs computer vision techniques to detect predefined hand gestures, which are mapped to cursor movements and mouse clicks. Voice commands are incorporated to extend functionality, such as opening files or performing system operations. Results indicated reasonable performance in terms of responsiveness and detection accuracy, with the system achieving effective control without specialized hardware. However, limitations include difficulties in distinguishing complex gestures, dependency on environmental lighting, and limited scalability across diverse user groups.

Devi, V. Anjana, E. Jahnavi, and R. Kavipriya [7] introduced a gesture-enabled virtual cursor system that integrates voice commands to enhance user interaction. Their approach combines hand gesture detection using computer vision with speech recognition for performing system operations. The authors reported improved user experience due to the naturalness of gestures combined with voice-based inputs, achieving acceptable latency and accuracy levels in real-time operation. A significant contribution of this work is its emphasis on multimodal integration, which compensates for individual weaknesses of gesture-only or voice-only approaches. Challenges such as misclassification of gestures in cluttered backgrounds and errors in voice recognition due to accent variations were acknowledged.

Shrivastava, Anurag, Sumit Pundir, Ashish Sharma, Aparna Srivastava, Ravindra Kumar, and Akhilesh Kumar Khan [8] proposed a system for virtual system control using hand gestures, designed to replace hardware-based input devices with a vision-based interaction approach. The methodology relies on real-time gesture recognition through image processing algorithms, where specific hand poses are associated with mouse events like movement, clicking, or scrolling. Limitations include difficulty in handling dynamic backgrounds, challenges in distinguishing similar gestures, and increased error rates under variable lighting. Despite these issues, the paper contributes to advancing gesture-based computing and highlights its potential in creating intuitive and contactless human-computer interfaces.

Tran, Dinh-Son, Ngoc-Huynh Ho, Hyung-Jeong Yang, Soo-Hyung Kim, and Guee Sang Lee [9] created a real-time virtual mouse system using fingertip sensing and RGB-D pictures. Their methodology employs depth cameras to capture three-dimensional data, allowing the system to accurately localize fingertips and map them to cursor movements. Experimental evaluations revealed high accuracy in fingertip tracking and stable cursor movement, with improved performance compared to traditional RGB-only methods. Despite these strengths, limitations include the need for specialized depth cameras, which increases system cost, and reduced efficiency under fast hand movements. The study demonstrates how depth sensing can significantly improve the reliability of gesture-based HCI systems.

Miah, Payel, Mirza Raina Gulshan, and Nusrat Jahan [10] explored mouse cursor control using eye gaze tracking as a hands-free interaction method. Their project captures gaze direction using a standard webcam and applies image processing algorithms to map eye position onto screen coordinates. Evaluation results demonstrated reasonable accuracy for basic cursor movements, though precision

decreased near screen edges. The authors highlighted the potential of such systems in assistive technology, particularly for individuals with motor impairments. Challenges identified include difficulties in distinguishing intentional from involuntary blinks, sensitivity to lighting changes, and the need for frequent calibration. The study adds to the body of research focusing on low-cost gaze-based interfaces for human-computer interaction.

III. METHODOLOGY

The overall workflow of the proposed system is illustrated in Fig 1. The process begins with capturing a live video stream from a standard webcam, followed by extraction of eye landmarks. These landmarks are analyzed for two primary tasks: gaze estimation and blink detection. Gaze direction is mapped to cursor movement, while blink events are interpreted as discrete mouse actions. Both components feed into the mouse control module, which integrates multiple interaction modes including cursor control, scrolling, and drag-and-drop. The system operates in real time without requiring specialized hardware.

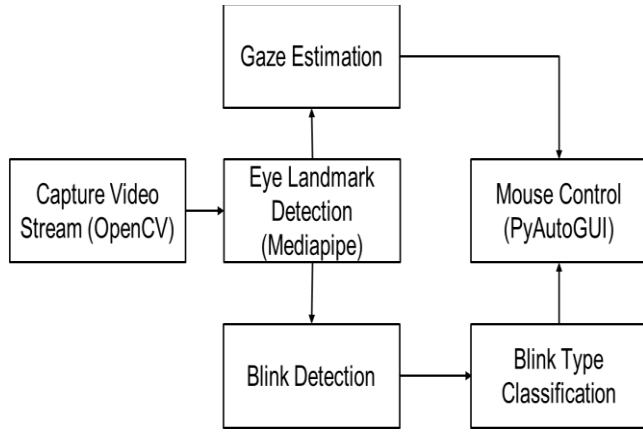


Fig 1. Architecture of proposed model

A. Capture Video Stream

The first step in the system pipeline involves capturing a continuous video stream using the OpenCV library. Each frame is converted into the appropriate format required by the MediaPipe Face Mesh model. Preprocessing tasks such as resizing, normalization, and color space conversion (BGR to RGB) are applied to ensure optimal model performance. The captured frames serve as the raw input for subsequent eye landmark extraction and analysis.

B. Gaze Estimation and Mouse Control

Gaze-based cursor control can be achieved using computer vision techniques without the need for specialized hardware. MediaPipe Face Mesh is used for eye landmark detection and tracking. The mouse cursor is controlled by PyAutoGUI in response to detected gaze movement.

1) *Mediapipe Face Mesh*: MediaPipe Face Mesh, developed by Google, is a deep-learning-based framework capable of detecting 468 facial landmarks from a single image. For this project, only a subset of landmarks corresponding to the eye contour and iris is utilized. To obtain stable gaze coordinates, the iris center is first computed as the average position of selected iris landmarks. This position is then normalized with respect to the bounding region of the eye, resulting in values between 0 and 1 for both horizontal and

vertical directions. This normalization ensures that gaze estimation remains consistent regardless of variations in face size or distance from the camera.

2) *PyAutoGUI*: Once gaze coordinates are obtained, they must be mapped to actual screen positions for cursor control. This mapping is achieved using the PyAutoGUI library, which provides cross-platform functions to control the mouse and keyboard programmatically.

Specifically, the normalized gaze coordinates (x_n, y_n) are transformed into screen coordinates (x_s, y_s) using the screen width and height as follows:

$$x_s = x_n \times \text{Screen Width}$$

$$y_s = y_n \times \text{Screen Height}$$

This linear mapping allows the full range of eye movement to correspond to the entire display area. To reduce jitter and sudden cursor jumps caused by small eye movements or noise in landmark detection, a smoothing mechanism is applied by averaging the current gaze position with previous frames. This results in more stable and user-friendly cursor movement.

PyAutoGUI functions such as `moveTo()`, `click()`, `scroll()`, and `dragTo()` are used to implement cursor movement and mouse actions. The continuous gaze estimation updates the cursor position in real time, while blink detection triggers discrete actions such as clicks.

C. Blink Detection

Blink detection approach is based on the Eye Aspect Ratio (EAR), which is a commonly used indicator of eye openness. Six important facial landmarks surrounding each eye are used to calculate the EAR, which records the eyelids' vertical and horizontal separations. It is defined as:

$$EAR = \frac{|p_2 - p_6| + |p_3 - p_5|}{2 \times |p_1 - p_4|}$$

where $p_1, p_2, \dots, p_6$ represent the coordinates that are extracted from the detected facial landmarks. For the left eye, the indices are [33, 160, 158, 133, 153, 144], and for the right eye [362, 385, 387, 263, 373, 380]. In this formula, the numerator represents the sum of vertical distances between the eyelid landmarks, and the denominator represents the horizontal eye width. The EAR value decreases significantly when the eye closes during a blink and returns to its baseline when the eye reopens.

Blinks are classified based on duration and frequency, allowing different mouse actions to be triggered as shown in Table 1.

TABLE I. DIFFERENT BLINK FEATURES

Blink Type	Mouse Feature
Short Blink (0.3s < x < 0.6s)	Left Click
Double Blink	Double Click
Long Blink (>0.6s)	Right Click

This classification ensures that involuntary blinks are ignored, while intentional blinks are recognized as control commands.

D. On-Screen Mode

A graphical interface is developed using Tkinter to enhance usability and allow the user to switch between different modes of operation. The interface presents buttons for enabling or disabling cursor control, scroll mode, and drag-and-drop mode.

1) *Cursor Control Mode*: In this mode, gaze estimation is directly mapped to cursor movement across the screen. Blinks act as click signals, with short blinks producing left clicks and long blinks producing right clicks. This mode serves as the default mode for general-purpose cursor navigation.

2) *Scroll Mode*: When scroll mode is activated, blinks no longer act as simple clicks. Instead, they control page scrolling. For Scrolling, we need to set scroll reference point using double blink which is marked by a cross symbol. When gaze moves above that point, the screen scrolls upward, and when gaze moves below that point, then the screen scrolls downward. Short blink can be used to pause and resume scrolling. The second double click exits scroll mode. A mapping of blink types to mouse functions in scroll mode is shown in Table II.

TABLE II. SCROLL MODE

Blink Type	Mouse Feature
Short Blink ($0.3s < x < 0.6s$)	Pause and resume scroll
Double Blink	Set Scroll Reference and exit Scroll Mode
Long Blink ($> 0.6s$)	Right Click

3) *Drag and Drop Mode*: When drag and drop mode is activated, the first short click initiates the dragging. The second short click is for dropping. This drag and drop mode can be used for various functions like moving files and folder, selecting text, etc. Double blink is used for exiting drag and drop mode. A mapping of blink types to mouse functions in drag and drop mode is shown in Table III.

TABLE III. DRAG AND DROP MODE

Blink Type	Mouse Feature
Short Blink ($0.3s < x < 0.6s$)	Drag and Drop
Double Blink	Exit Scroll Mode
Long Blink ($> 0.6s$)	Right Click

E. Evaluation Metrics

Evaluation of the proposed system will be carried out using a three-pronged framework consisting of task-based performance testing, system performance metrics, and user feedback analysis. This pilot study consists is conducted using

a group of 20 participants. This evaluation approach ensures both the technical validity and user-centered effectiveness of the proposed system.

1) *Task-Based Performance Testing*: The participants are asked to do a set of task and certain metrics are noted. This will provide a direct measure of usability and efficiency. The tasks are:

- Task 1*: Create a new folder (Right click → New → Folder).
- Task 2*: Navigate to a webpage and scroll up and down.
- Task 3*: Perform drag-and-drop (e.g., moving a folder or selecting and dragging text).

The metrics that are recorded are:

- Completion Time* (time taken in seconds for each task).
- Error Rate* (number of failed attempts, e.g., unintended clicks, missed drags, or mis-scrolling).

2) *System Performance Metrics*: These metrics will help assess the technical robustness of the system under varying lighting and background conditions.

a) *Blink Detection Accuracy*: For evaluating blink detection accuracy, each participant performs short blink (left click), long blinks (right click), double blink (double click) 10 times continuously. Number of successful detection and classification for each blink is recorded for calculating accuracy.

$$\text{Accuracy (\%)} = \frac{\text{Successful detections}}{\text{Total attempts}} \times 100$$

b) *False Activation Rate*: During normal operation, record the number of false clicks triggered by involuntary blinks. This is important to measure system reliability.

3) *User Experience Feedback*: Participants complete a post-test questionnaire using a 5-point Likert scale (1 = strongly disagree, 5 = strongly agree). This subjective feedback will complement the quantitative results, providing a holistic evaluation of system performance.

IV. RESULT

The performance of the proposed eye-gaze controlled virtual mouse was evaluated through task completion experiments, system-level accuracy measurements, and usability feedback from participants. The evaluation framework aimed to measure both objective performances, such as task completion time and blink detection accuracy, and subjective factors, such as user comfort and satisfaction. Results are presented in three parts: and supported with charts for clear interpretation.

1) *Task-Based Performance Testing*: Participants were asked to complete three tasks—folder creation, webpage scrolling, and drag-and-drop operations—and the average completion time and error count were recorded for each task. The aggregated results are shown in Table IV.

TABLE IV. TASK-BASED PERFORMANCE METRICS

Metrics	Task 1	Task 2	Task 3
Completion Time in seconds (avg)	44.1	22.5	20.65
Error rate (avg)	10.9	3.8	4.1



Fig 2. Task-based performance metrics

Task 1 (folder creation), which required multiple sequential actions including a right-click and menu navigation, recorded the highest average completion time and error rate. This is expected, as multi-step interactions are more sensitive to blink timing and precision. Task 2 (scrolling) demonstrated the best performance, with low completion time and very few errors, reflecting the strength of gaze-based input for continuous operations. Task 3 (drag-and-drop) performed moderately well, with completion time and error levels between the other two tasks. These findings indicate that the system performs strongly for continuous controls like scrolling, while precision tasks involving multiple blink activations can introduce additional complexity and overhead.

2) *System Performance Metrics*: Blink detection accuracy and click classification accuracy was evaluated by instructing each participant to perform multiple instances of short blinks, long blinks, and double blinks. The overall average accuracies recorded across all participants are shown in Table V.

TABLE V. BLINK DETECTION ACCURACY

Function	Accuracy (%)
Left click	81
Right click	84
Double click	70

The results show that right and left blinks achieve relatively high detection accuracy, while double blinks are more challenging due to tighter timing constraints between consecutive blinks. This aligns with the user feedback, where many participants noted that double blinks required more concentration and were harder to execute consistently.

False activation analysis further revealed that unintentional clicks were not observed when lighting was adequate and the participant's face was clearly visible. However, in poor lighting conditions, when the face was

partially obscured, or when the head was turned sideways, false detections and misclassifications increased. This indicates that system reliability is affected by environmental and tracking conditions.

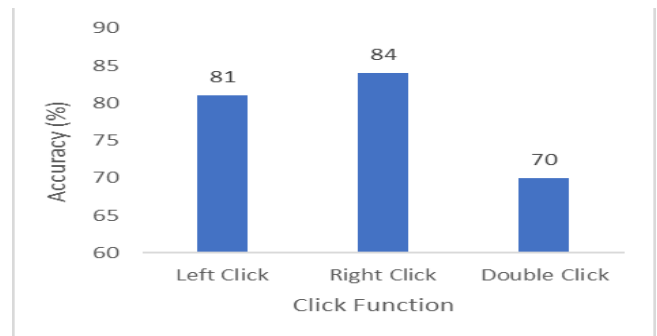


Fig 3. Blink detection and click classification accuracy

3) *User Experience Feedback*: A post-test user experience survey was conducted using a 5-point Likert scale to evaluate multiple aspects of user experience. The average ratings obtained from the participants are shown in Table VI.

TABLE VI. USER EXPERIENCE FEEDBACK

Q.no.	Questions	Average Rating
1	Ease of use	3.65
2	Ease of learning	4.1
3	Eye comfort while using the system	3.65
4	Satisfaction with cursor movement	4.3
5	Satisfaction with scroll mode	3.9
6	Satisfaction with drag-and-drop mode	4.15
7	Overall satisfaction of the system	4.25
8	Which blink type was most difficult (Single/Double/Long)	Double Blink

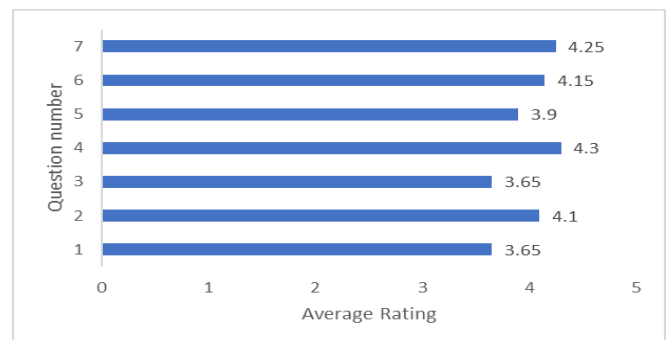


Fig 4. User experience feedback

The high ratings for ease of learning indicate that users were able to understand and use the system quickly. Cursor movement and drag-and-drop mode received strong satisfaction scores, reflecting good responsiveness and control accuracy. Eye comfort received a slightly lower rating, suggesting mild visual strain during prolonged use.

Participants also identified double blinks as the most difficult blink type to perform, confirming the quantitative findings from the system performance analysis. Overall, the user study demonstrates that the proposed system is well-received, intuitive to use, and effective for real-time human–computer interaction.

Despite the overall positive performance, certain limitations were observed. The system is sensitive to lighting conditions, with reduced accuracy in low or uneven illumination. Variations in head pose, such as tilting or turning away from the camera, can also affect gaze estimation and landmark detection. Additionally, prolonged use may cause mild eye strain, and interactions like double blink detection require higher precision from users. These factors indicate the need for improved robustness through adaptive calibration and enhanced tracking techniques.

V. CONCLUSION

The presented system demonstrates an effective and affordable approach for controlling a computer mouse using eye-gaze tracking and blink-based commands. By integrating computer vision techniques with MediaPipe Face Mesh for gaze estimation, EAR-based blink detection, and PyAutoGUI for cursor operations, the system successfully replaces traditional mouse functionalities. Evaluation results indicate that the system performs well for tasks such as cursor navigation, scrolling and drag-and-drop. User experience feedback further confirms that the system is intuitive, easy to learn, and generally comfortable for short to medium usage durations. The results demonstrate how gaze-based interfaces have the potential to offer a human-computer interaction paradigm that is accessible, particularly for people with motor disabilities or in hands-free computing situations.

The system's dependability and adaptability can be improved in a number of ways for upcoming projects. First, optimization of gaze movement algorithms can minimize jitter and improve cursor stability, particularly for high-precision tasks. Second, incorporating voice-based monitoring and control could provide a multimodal interaction framework. Third, adaptive calibration techniques could be introduced to automatically adjust sensitivity based on user-specific eye dynamics and environmental lighting conditions, reducing the need for manual tuning. In addition, extending the system for mobile and wearable platforms, or integrating it with augmented and virtual reality environments, could further expand its applicability. These enhancements would allow the system to evolve into a robust, user-friendly, and universally accessible interaction framework.

VI. REFERENCE

- [1] Waideman, Bruno, and Plínio Thomaz Aquino. "Augmentative and Alternative Communication Using Eye Tracking and Word Recommendation Using Language Models." *IEEE Latin America Transactions* 23, no. 8 (2025): 637-645.
- [2] Lin, Yi, Xingwei Chen, Chuyang Zhang, and Jingjing Chen. "A virtual object manipulation method based on eye-gaze selection and head-eye coordination movement." *International Journal of Human-Computer Studies* 197 (2025): 103458.
- [3] Beevi, L. Sherin, PM Joe Prathap, and W. Vinil Dani. "Revolutionizing Mouse Cursor Control for Enhanced User Interaction." In *2025 International Conference on Visual Analytics and Data Visualization (ICVADV)*, pp. 1415-1420. IEEE, 2025.
- [4] Sai, K. Chenchu Naga, S. Shargunam, K. Surendra Reddy, K. Bheemendra, and K. Santhosh Reddy. "Virtual Mouse using Hand Gestures-Eye Speech Recognition." In *2024 5th International Conference on Data Intelligence and Cognitive Informatics (ICDICI)*, pp. 1459-1467. IEEE, 2024.
- [5] Dharavath, Krishna, Ganguru Manoj Kumar, Kallem Rishitha Reddy, and Mandla Hemanth Reddy. "Gesture and Voice Controlled Virtual Mouse for Elderly People." In *2024 2nd International Conference on Networking and Communications (ICNWC)*, pp. 1-5. IEEE, 2024.
- [6] Morajkar, Aarti, Atheena Mariyam James, Minoli Bagwe, Aleena Sara James, and Aruna Pavate. "Hand gesture and voice-controlled mouse for physically challenged using computer vision." *Advanced engineering days (AED)* 6 (2023): 127-131.
- [7] Devi, V. Anjana, E. Jahnavi, and R. Kavipriya. "Enhancing User Interaction: Gesture-Enabled Virtual Cursor with Voice Integration." In *2024 4th International Conference on Pervasive Computing and Social Networking (ICPCSN)*, pp. 279-286. IEEE, 2024.
- [8] Shrivastava, Anurag, Sumit Pundir, Ashish Sharma, Aparna Srivastava, Ravindra Kumar, and Akhilesh Kumar Khan. "Control of A Virtual System with Hand Gestures." In *2023 3rd International Conference on Pervasive Computing and Social Networking (ICPCSN)*, pp. 1716-1721. IEEE, 2023.
- [9] Tran, Dinh-Son, Ngoc-Huynh Ho, Hyung-Jeong Yang, Soo-Hyung Kim, and Guee Sang Lee. "Real-time virtual mouse system using RGB-D images and fingertip detection." *Multimedia tools and applications* 80, no. 7 (2021): 10473-10490.
- [10] Miah, Payel, Mirza Raina Gulshan, and Nusrat Jahan. "Mouse cursor movement and control using eye gaze-a human computer interaction." In *2022 International Conference on Artificial Intelligence of Things (ICAIoT)*, pp. 1-6. IEEE, 2022.
- [11] Singh, Jaiteg, and Nandini Modi. "A robust, real-time camera-based eye gaze tracking system to analyze users' visual attention using deep learning." *Interactive Learning Environments* 32, no. 2 (2024): 409-430.
- [12] Veluchamy, S. "Deep Transfer Learning Enabled Eye Gaze and Blink-Driven Support System for Paralysed Patients." In *2025 3rd International Conference on Advancement in Computation & Computer Technologies (InCACCT)*, pp. 784-789. IEEE, 2025.
- [13] Roy, Koushik, and Dibalohe Chanda. "A robust webcam-based eye gaze estimation system for human-computer interaction." In *2022 International Conference on Innovations in Science, Engineering and Technology (ICISSET)*, pp. 146-151. IEEE, 2022.
- [14] Molina-Cantero, Alberto J., Clara Lebrato-Vázquez, Juan A. Castro-García, Manuel Merino-Monge, Félix Biscarri-Triviño, and José I. Escudero-Fombuena. "A review on visible-light eye-tracking methods based on a low-cost camera." *Journal of Ambient Intelligence and Humanized Computing* 15, no. 4 (2024): 2381-2397.
- [15] Li, Yujie, Jiahui Chen, Jiaxin Ma, Xiwen Wang, and Wei Zhang. "Gaze estimation based on convolutional structure and sliding window-based attention mechanism." *Sensors* 23, no. 13 (2023): 6226.
- [16] Kanakaprabha, S., P. Arulprakash, S. Praveena, G. Preethi, Tahsin Sakkeer Hussain, and V. Vignesh. "Mouse cursor controlled by eye movement for individuals with disabilities." In *2023 7th International Conference on Intelligent Computing and Control Systems (ICICCS)*, pp. 1750-1758. IEEE, 2023.
- [17] Padma, R., Suchitra T. Yadav, R. J. Keerthana, U. S. Tejeshwari, and Amulya Kenche. "Detecting the Eye Movement to Control the Cursor Movement of the system." In *2024 4th International Conference on Data Engineering and Communication Systems (ICDECS)*, pp. 1-7. IEEE, 2024.
- [18] Karatay, Ramazan, Burak Demir, Ali Arda Ergin, and Erdem Erkan. "A real-time eye movement-based computer interface for people with disabilities." *Smart Health* 34 (2024): 100521.
- [19] Kwak, SeHyun, Daeho Lee, Siwon Kim, and Junghoon Park. "A Real-Time Eye Gaze Tracking Based Digital Mouse." In *International Conference on Innovative Mobile and Internet Services in Ubiquitous Computing*, pp. 39-46. Cham: Springer Nature Switzerland, 2024.
- [20] Wang, Xinming, Jianhua Zhang, Hanlin Zhang, Shuwen Zhao, and Honghai Liu. "Vision-based gaze estimation: A review." *IEEE Transactions on Cognitive and Developmental Systems* 14, no. 2 (2021): 316-3